

Reviewer Pack — Minimal Order of p-Adic Points over Affine Schemes and AC0 C...

Entry e0fb9ecbeadc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 07:13:52 UTC

Minimal Order of p-Adic Points over Affine Schemes and AC0 Circuit Threshold for Modular Functions

Entry ID: e0fb9ecbeadc **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 07:13:52 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): Boolean Circuit Complexity

Statement:

[‘For any affine scheme X defined over a finite field F , the minimal order of a p-adic point on X is at most $c \cdot \log(n)$ for any AC0 circuit C computing a modular function with n inputs.’, ‘Equivalently, for all circuits C in AC0 that compute a modular function, the minimal order of a p-adic point on the algebraic variety defined by C is bounded above by $c \cdot \log(n)$.’, ‘If C computes the parity function, then the conjecture implies that there exists a p-adic point of order at most $c \cdot \log(n)$ on the variety defined by C .’]

Rationale (proposer’s reasoning):

[‘p-adic analysis has been used to study Diophantine equations and algebraic varieties, which are closely related to boolean functions and circuits.’, ‘The minimal order of a p-adic point over an affine scheme can provide insights into the complexity of computing modular functions using boolean circuits.’, ‘This conjecture may reveal new connections between number theory and computational complexity.’]

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9de20af4d5713498

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all AC0 circuits C computing modular functions, the minimal order of a p-adic point on the algebraic variety defined by C is less than or equal to $c \cdot \log(n)$ with at least 90% of seeds producing such points. The conjecture is falsified if any seed produces a point with a minimal order greater than $c \cdot \log(n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -p-adic analysis AND Boolean circuit complexity-AC0 circuit threshold modular function AND p-adic points-minimal order p-adic points affine scheme AC0 circuit

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ac0_circuit(n):
        # Simplified AC0 circuit generation for modular functions
        return [random.choice([0, 1]) for _ in range(2**n)]

    def find_p_adic_point(circuit):
        # Placeholder function to simulate finding a p-adic point
        # This is a dummy implementation and should be replaced with actual logic
        return random.randint(1, 10)

    def log_n(n):
        if n <= 0:
            return float('inf')
        return math.log(n)

    c = 2 # Placeholder constant for the conjecture

    results = []
    for _ in range(30): # Test with 30 random seeds
        n = random.randint(5, 40) # Sweep n from 5 to 40
        circuit = generate_ac0_circuit(n)
        p_adic_order = find_p_adic_point(circuit)
        log_n_value = log_n(n)

        results.append({
            "n": n,
```

```

        "c_log_n": c * log_n_value,
        "p_adic_order": p_adic_order
    })

    all_holds = True
    for result in results:
        if result["p_adic_order"] > result["c_log_n"]:
            all_holds = False

    return {
        "metric_name": "minimal_p_adic_order",
        "metric_value": sum(result["p_adic_order"] for result in results) / len(results),
        "instances_tested": len(results),
        "conjecture_holds": all_holds,
        "counterexample": "" if all_holds else "circuit_size={}".format(max(result["n"] for result in results))
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    total_metric_value = 0
    total_instances_tested = 0
    conjecture_holds_count = 0

    for seed in seeds:
        trial_result = run_trial(seed)
        print("TRIAL: {}".format(trial_result))

        total_metric_value += trial_result["metric_value"]
        total_instances_tested += trial_result["instances_tested"]
        if trial_result["conjecture_holds"]:
            conjecture_holds_count += 1

    mean_metric_value = total_metric_value / len(seeds)
    support_fraction = conjecture_holds_count / len(seeds)

    if all(trial_result["conjecture_holds"] for trial_result in [run_trial(seed) for seed in seeds]):
        print("RESULT: SUPPORTED mean={} std={} support_fraction={}".format(mean_metric_value, 0, support_fraction))
    elif any(not trial_result["conjecture_holds"] for trial_result in [run_trial(seed) for seed in seeds]):
        first_failing_seed = next(seed for seed in seeds if not run_trial(seed)["conjecture_holds"])
        print("RESULT: FALSIFIED counterexample=\"circuit_size\" first_failing_seed={}".format(first_failing_seed))
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture's support or falsification. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15176
2	preregistration	ollama_remote	glm4:latest	0	0	10133
3	novelty	ollama_remote	glm4:latest	0	0	8175
4	novelty	ollama_remote	glm4:latest	0	0	9069
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20478
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9572
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7324
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8491
9	judge	ollama_remote	glm4:latest	0	0	16843

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 105260 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/e0fb9ecbeadc.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/e0fb9ecbeadc.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/e0fb9ecbeadc.tar.gz* (if generated)